



YENEPOYA UNIVERSITY

HIGH LEVEL DESIGN (HLD) DOCUMENT ON Edge AI for Healthcare Realtime Monitoring

Student Name:	Rhidhik KS
Registration No:	23BCAICD092
Campus ID:	25287
Industry Mentor:	Sumit Kumar Shukla
Guided By:	Ms. Deeksha Shetty

HIGH LEVEL DESIGN

ICU Monitoring System

using Edge AI & Firebase Cloud

Document Type	High Level Design (HLD)
Version	1.0
Date	April 2026
Domain	Healthcare / IoT / Edge AI

Table of Contents

1. Introduction.	6
1.1. Scope of the document.	6
1.2. Intended Audience	6
1.3. System overview.	6
2. System Design.	6
2.1. Application Design	6
2.2. Process Flow.	6
2.3. Information Flow.	7
2.4. Components Design	8
2.5. Key Design Considerations	8
2.6. API Catalogue.	8
3. Data Design.	8
3.1. Data Model	8
3.2. Data Access Mechanism	9
3.3. Data Retention Policies	9
3.4. Data Migration	9
4. Interfaces	9
5. State and Session Management	10
6. Caching	10
7. Non-Functional Requirements	10
7.1. Security Aspects	10
7.2. Performance Aspects	10
8. References	11

1. Introduction

1.1 Scope of the Document

This High Level Design (HLD) document describes the overall system architecture, design approach, and technology decisions for the ICU Monitoring System using Edge AI and Firebase Cloud. It covers the end-to-end workflow from hardware sensor data collection through edge AI processing to cloud storage and real-time dashboard visualization.

This document is intended to serve as the primary architectural reference for development, testing, and deployment activities related to the ICU Monitoring System.

1.2 Intended Audience

- Software Architects and System Designers
- Backend and Frontend Developers
- IoT and Embedded Systems Engineers
- Project Managers and Technical Leads
- Academic Reviewers and Industry Mentors

1.3 System Overview

The ICU Monitoring System is a hybrid Edge-Cloud AI platform designed to continuously monitor environmental conditions (temperature and humidity) in an Intensive Care Unit setting. The system employs a multi-layered architecture:

- Sensor Layer: DHT11/DHT22 sensors connected to Arduino Uno to capture real-time environmental data
- Edge Layer: Python-based edge monitor reads serial data from Arduino, runs a TensorFlow Lite model for on-device AI inference, and pushes results to Firebase
- Cloud Layer: Firebase Realtime Database stores structured patient data accessible globally
- Application Layer: Streamlit dashboard renders live graphs, AI risk predictions, and visual alerts for all monitored patients

The system supports multi-patient monitoring (P001, P002, P003) with three alert states: SAFE, WARNING, and CRITICAL, delivered via both hardware LED and software dashboard.

2. System Design

2.1 Application Design

2.1.1 Layered Architecture

The system follows a five-layer architecture that cleanly separates concerns across hardware, edge processing, cloud, and presentation:

Layer	Component	Technology	Responsibility
L1 – Sensor	DHT11/DHT22 + Arduino Uno	C/C++ (Arduino IDE)	Read temperature & humidity; transmit via Serial @ 9600 baud
L2 – Edge	edge_monitor.py	Python, PySerial, TFLite	Parse serial data; AI inference; push to Firebase
L3 – AI Model	model.tflite	TensorFlow Lite	Binary risk classification (SAFE / WARNING)
L4 – Cloud	Firebase RTDB	Google Firebase	Real-time structured storage; REST and SDK access
L5 – App	dashboard.py	Streamlit, Plotly, Pandas	Live visualization; alerts; multi-patient view

2.2 Process Flow

The end-to-end data flow follows a pipeline from physical sensor readings through AI inference and cloud synchronization to visual presentation:

- Step 1 – Sensor Reading: Arduino reads temperature and humidity from DHT11 sensor every 2 seconds
- Step 2 – Serial Transmission: Data is sent as CSV string (e.g., '28.5,65.2') over Serial port at 9600 baud
- Step 3 – Edge Parsing: edge_monitor.py reads the serial line using PySerial and splits the CSV values
- Step 4 – AI Inference: Normalized input [temp/50.0, hum/100.0] is fed into the TFLite model; output score is returned
- Step 5 – Status Classification: Rule-based override determines CRITICAL (temp>38 or hum>80), WARNING (temp>30 or hum>70), or SAFE
- Step 6 – Firebase Upload: Patient record (temp, humidity, ai_score, status, timestamp) is written to Firebase RTDB under patients/{patient_id}
- Step 7 – Dashboard Fetch: Streamlit dashboard auto-refreshes every 1 second and reads all patient nodes from Firebase
- Step 8 – Visualization: Patient cards, trend graphs (Plotly), and status alerts are rendered in real-time

2.3 Information Flow

Data flows unidirectionally from sensors to the cloud; the dashboard reads from the cloud only. There is no feedback loop from the dashboard to the hardware in the current version.

Data Source	DHT11/DHT22 sensor on Arduino Uno (Digital Pin 2)
Serial Protocol	UART at 9600 baud, CSV format: 'temperature,humidity\n'
Edge Processing	PySerial read → CSV split → TFLite inference → status rule engine
Firebase Path	patients/{P001 P002 P003} → {temp, humidity, ai_score, status, time}
Dashboard Read	firebase_admin db.reference('patients').get() polled every 1 second
Alert Propagation	Hardware: LED on Pin 13 lights when temp>30 or hum>60. Software: card color and CSS blink animation

3. Components Design

3.1 Component Overview

Component	File / Module	Technology	Description
Arduino Firmware	patient_monitoring.ino	C++ / Arduino IDE	Reads DHT sensor, serializes CSV, controls LED
Edge Monitor	edge_monitor.py	Python 3, PySerial	Serial reader, TFLite runner, Firebase writer
AI Model (Keras)	train_model.py / model.h5	TensorFlow / Keras	Binary classification model trained on temp+humidity
AI Model (TFLite)	model.tflite	TensorFlow Lite	Quantized model for low-latency edge inference
Model Converter	convert.py	TensorFlow	Converts .h5 to quantized .tflite format
Dashboard	dashboard.py	Streamlit, Plotly	Real-time multi-patient monitoring UI
Cloud Database	Firebase RTDB	Google Firebase	Structured NoSQL real-time database
Local Backup	health_log.csv	CSV file	Local data store for offline analysis (future use)

3.2 Key Design Considerations

3.2.1 Edge AI Over Cloud AI

The AI inference is intentionally placed at the edge (local Python process) rather than in the cloud. This provides:

- Latency: Inference completes in <5ms on local hardware vs. 100–300ms cloud round-trip
- Resilience: Monitoring continues during intermittent internet connectivity
- Privacy: Raw sensor data is not required to leave the local network for inference

3.2.2 Model Design

The TensorFlow Lite model is a compact 3-layer dense neural network:

- Input: 2 normalized features — temperature (divided by 50) and humidity (divided by 100)
- Architecture: Dense(16, ReLU) → Dense(8, ReLU) → Dense(1, Sigmoid)
- Output: Continuous risk score in [0.0, 1.0]; higher values indicate greater risk
- Quantization: DEFAULT optimization applied during TFLite conversion to reduce model size (~2.8KB)

3.2.3 Status Classification Logic

Status determination combines the AI model output with deterministic threshold rules:

- CRITICAL: Temperature > 38°C OR Humidity > 80% (immediate hardware and visual alert)
- WARNING: Temperature > 30°C OR Humidity > 70% (visual warning on dashboard)
- SAFE: All values within normal range (green status card)

3.2.4 Multi-Patient Simulation

Three patient IDs (P001, P002, P003) are cycled through a round-robin assignment in the edge monitor. Each serial reading is attributed to the next patient in sequence. This simulates a 3-bed ICU ward with a single sensor setup for demonstration.

4. API Catalogue

4.1 Firebase Realtime Database API

The system uses the Firebase Admin SDK (Python) for all cloud operations. No custom REST API is exposed.

Operation	SDK Method	Firebase Path	Payload / Return	Description
Write Patient Data	ref.set({})	patients/{patient_id}	{temp, humidity, ai_score, status, time}	Overwrites current patient record (latest reading)
Read All Patients	ref.get()	patients/	Dict of all patient nodes	Dashboard fetches all patients in one call
Initialize App	firebase_admin.initialize_app()	N/A	App credentials	Authenticates using Service Account JSON key

4.2 Internal Python Interfaces

Interface	Module	Call	Input	Output
Serial Read	edge_monitor.py	ser.readline()	None	Raw bytes decoded to CSV string
TFLite Inference	edge_monitor.py	interpreter.invoke()	Float32 array [1x2]	Risk score Float32
Dashboard Refresh	dashboard.py	st_autorefresh(1000)	None	Forces Streamlit re-run every 1s

5. Data Design

5.1 Data Model

The Firebase Realtime Database stores patient data in a flat JSON tree structure:

Field	Type	Example Value	Description
temp	Float	28.5	Temperature reading in degrees Celsius from DHT sensor
humidity	Float	65.2	Relative humidity percentage from DHT sensor
ai_score	Float	0.73	TFLite model output: risk probability score [0.0 – 1.0]
status	String	WARNING	Derived status: SAFE WARNING CRITICAL
time	String (ISO)	2026-04-22 10:30:45	Timestamp from Python datetime.now() at time of upload

5.2 Data Access Mechanism

All data access is mediated through the Firebase Admin SDK authenticated with a Service Account JSON key (firebase_key.json). The edge monitor writes data using set() which replaces the patient node entirely on each reading cycle. The dashboard reads all patient data in a single get() call on every auto-refresh cycle (every 1 second).

5.3 Data Retention Policies

In the current version, Firebase only stores the latest reading for each patient (last-write-wins via ref.set()). There is no historical data retention in the cloud. A local health_log.csv file provides a flat backup for post-analysis. Future versions should implement timestamped child nodes for historical retention.

5.4 Data Migration

No formal data migration is required for the current version since the system is designed for ephemeral real-time monitoring. Archived CSV data can be ingested into any analytics platform (pandas, Excel, Tableau) for retrospective analysis. If the Firebase schema evolves to include historical nodes, a migration script will be required.

6. Interfaces

6.1 Hardware Interfaces

Arduino – Sensor	DHT11/DHT22 on Digital Pin 2 (5V logic); polling every 2 seconds using DHT library
Arduino – LED	LED connected to Digital Pin 13; HIGH when temp>30°C or hum>60%
Arduino – Serial	UART TX on USB port; CSV encoded string 'temp,humidity\n' at 9600 baud
PC – Arduino	USB Serial connection; PySerial opens port COM3 (configurable) at 9600 baud

6.2 Software Interfaces

edge_monitor.py → Firebase	firebase_admin SDK; authenticated with Service Account; writes to patients/{id}
dashboard.py → Firebase	firebase_admin SDK; reads entire patients/ subtree via ref.get()
edge_monitor.py → TFLite	tf.lite.Interpreter; allocate_tensors(); set_tensor(); invoke(); get_tensor()
dashboard.py → Streamlit	st.markdown(), px.line(), st.columns(), st.selectbox(), st.autorefresh()

6.3 User Interface

The Streamlit dashboard provides a web-based UI accessible at localhost:8501 (default). Key UI elements include:

- Patient Cards: Color-coded status cards (green/yellow/red with blink animation for CRITICAL)
- Temperature Graph: Multi-line Plotly chart showing temperature trend for all patients
- Humidity Graph: Multi-line Plotly chart showing humidity trend for all patients
- Patient Selector: Dropdown to isolate and view details for a specific patient

7. State and Session Management

7.1 Edge Monitor State

The edge monitor maintains minimal stateful variables between loop iterations:

- `patients[]`: Static list of patient IDs [P001, P002, P003]
- `index`: Integer counter cycling through patient IDs in round-robin fashion
- `ser`: Serial port object maintained for the process lifetime
- `interpreter`: TFLite Interpreter object allocated once and reused each cycle

7.2 Dashboard Session State

Streamlit rerenders the entire page on each refresh cycle. No server-side session persistence is maintained. Firebase acts as the shared state store between the edge monitor and the dashboard. The auto-refresh mechanism (`st_autorefresh` at 1000ms) drives the read cycle.

8. Caching

The current version does not implement explicit caching. Firebase provides its own internal caching for RTDB nodes. Future enhancements may include:

- In-memory circular buffer (deque) in `dashboard.py` to accumulate real historical readings for trend graphs (currently fake random noise is used)
- Redis or in-process cache for the edge monitor to batch Firebase writes and reduce API calls

9. Non-Functional Requirements

9.1 Security Aspects

Authentication	Firestore access requires a Service Account key (firebase_key.json). This file must never be committed to source control.
Authorization	Firestore Realtime Database rules should restrict write access to authenticated service accounts only and read access to the dashboard client.
Data Transmission	All Firestore traffic is encrypted over HTTPS/TLS by default.
Serial Port	Physical access to the Arduino serial port is required to intercept sensor data; no additional encryption is applied at the hardware layer.
Secrets Management	firebase_key.json should be stored as an environment variable or secrets manager entry in production deployments.

9.2 Performance Aspects

Sensor Sampling Rate	One reading every 2 seconds (DHT11 maximum stable rate)
TFLite Inference Latency	< 5ms on standard laptop hardware (quantized INT8 model)
Firestore Write Latency	Typically 50–200ms depending on network conditions
Dashboard Refresh Rate	1-second auto-refresh; limited by Firestore read latency
Model Size	~2.8KB (quantized TFLite) — suitable for microcontroller deployment in future
Scalability	Current design supports 3 patients; extending to N patients requires only adding patient IDs to the round-robin list and provisioning Firestore nodes

9.3 Reliability

- The edge monitor wraps all serial and Firestore operations in a try/except block to prevent crashes on transient errors
- DHT sensor NaN checks prevent garbage data from propagating through the pipeline
- Firestore RTDB provides 99.95% SLA uptime for cloud data availability

10. References

Arduino DHT Library	https://github.com/adafruit/DHT-sensor-library
TensorFlow Lite Python API	https://www.tensorflow.org/lite/api_docs/python
Firebase Admin Python SDK	https://firebase.google.com/docs/admin/setup#python
Firebase Realtime Database	https://firebase.google.com/docs/database
Streamlit Documentation	https://docs.streamlit.io
Plotly Express	https://plotly.com/python/plotly-express/
PySerial Documentation	https://pyserial.readthedocs.io
TFLite Model Optimization	https://www.tensorflow.org/model_optimization